

UNIDAD DE TRABAJO 02.3

React con TypeScript

Tipos, compilador y componentes fuertemente tipados

```
type Props = {  
  type?: 'information' | 'warning';  
  heading: string;  
  children: ReactNode;  
  closable?: boolean;  
  onClose?: () => void;  
};
```

Qué veremos en este tema

01

Ventajas de TypeScript

Qué aporta frente a JavaScript

02

Tipos básicos

Anotaciones, inferencia, any, unknown

03

Crear tipos

Alias, uniones, interfaces, as const

04

El compilador

tsc y tsconfig.json

05

React + TypeScript

Props, estado y eventos

Al terminar, sabrás tipar las **props**, el **estado** y los **eventos** de un componente de React.

01

Ventajas de TypeScript

Qué es, cómo se relaciona con JavaScript y por qué nos hace más productivos.

Un superconjunto de JavaScript



El navegador solo entiende JavaScript: el código TypeScript se **transpila** antes de ejecutarse.



Se usa en el *frontend* (React, Angular, Vue) y en el *backend* (Node.js, Bun, Deno).

Novedades de 2026

6.0

MARZO 2026 · VERSIÓN «PUENTE»

- **strict** activado por defecto
- Destino por defecto: ECMAScript más reciente
- Declara obsoletas opciones antiguas

7.0

JULIO 2026 · COMPILADOR NATIVO

- Compilador reescrito en **Go**
- Compilaciones **8–12× más rápidas**
- Las opciones obsoletas ya dan error

Detección temprana de errores

```
function calculateTotalPrice(  
  product: { name: string; unitPrice: number },  
  quantity: number,  
  discount: number,  
) {  
  const priceWithoutDiscount =  
    product.price * quantity; // error  
  const discountAmount =  
    priceWithoutDiscount * discount;  
  return priceWithoutDiscount - discountAmount;  
}
```

El error

La propiedad **price** no existe en **product**: la correcta es **unitPrice**.

En JavaScript...

...el mismo código no avisa: el fallo aparece en ejecución, o en producción.

IntelliSense

Con tipos, el editor sugiere las propiedades reales al escribir **product**.

JavaScript: tipado dinámico... y débil

Tipado dinámico: el tipo va con el valor, no con la variable.

```
let score = 9;
typeof score; // 'number'
score = 'ten';
typeof score; // 'string'
```

Tipado débil: conversiones implícitas de tipo.

```
'5' * 2; // 10
'5' + 2; // '52'
const date = new Date();
typeof date; // 'object'
date instanceof Date; // true
```

Date sí existe en JavaScript: **typeof** solo distingue unos pocos primitivos y devuelve **object** para cualquier objeto.

02

Tipos básicos

Anotaciones, inferencia y los tipos especiales `any`, `unknown` y `never`.

Anotaciones e inferencia de tipos

Anotación explícita

```
let unitPrice: number;  
unitPrice = 'Table'; // error  
  
function getTotal(  
  unitPrice: number,  
  quantity: number,  
  discount: number,  
): number { ... }
```

Inferencia

```
let flag = false; // boolean  
flag = 'table'; // error  
  
let today = new Date(); // Date  
today.addMonths(2); // error  
  
const numbers = [1, 2, 3]; // number[]
```

Regla práctica: **confía en la inferencia** y anota solo cuando el tipo inferido no es el que quieres.

any, unknown y never

unknown

tipo superior: acepta cualquier valor



`string · number · boolean ·
objetos...`



never

tipo inferior: ningún valor

any

Desactiva la comprobación de tipos. Evítalo.

unknown

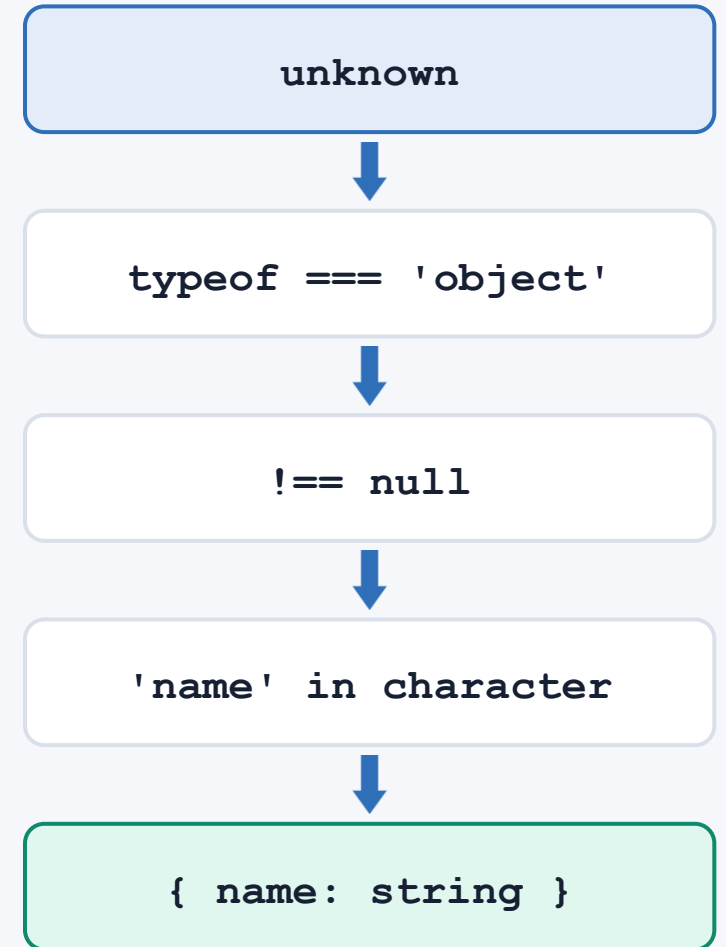
Admite cualquier valor, pero no deja usarlo hasta comprobar qué es.

never

No admite ningún valor. Ej.: una función que siempre lanza un error.

Estrechar el tipo con guardas de tipo

```
function isCharacter(  
  character: unknown,  
) : character is { name: string } {  
  return (  
    typeof character === 'object' &&  
    character !== null &&  
    'name' in character &&  
    typeof character.name === 'string'  
  );  
}  
  
if (isCharacter(data)) {  
  console.log(data.name); // ✓  
}
```



03

Crear nuestros propios tipos

Tipos de objeto, alias, intersecciones, uniones, interfaces y objetos constantes.

Tipos de objeto y alias de tipo

```
// Tipo de objeto en línea; ? = opcional
const table: { name: string; unitPrice?: number } = {
  name: 'Table',
};

// Alias de tipo: le damos un nombre reutilizable
type Product = { name: string; unitPrice?: number };

let chair: Product = { name: 'Chair', unitPrice: 40 };
chair.discount = 10; // error
```

?

Propiedad o parámetro opcional.

type

Da nombre a un tipo para no repetirlo.

Inferencia

También funciona con objetos.

Intersecciones y tipos de función

```
type Purchase = (quantity: number) => void;

type Product = {
  name: string;
  unitPrice?: number;
  purchase: Purchase;
};

// Intersección: Product + discount
type DiscountedProduct = Product & { discount: number };

let table: Product = {
  name: 'Table',
  purchase: (quantity) => console.log(`Purchased ${quantity} tables`);
};
```

&

Combina tipos: **tipo de intersección**.

void

La función no devuelve ningún valor.

Inferido

quantity toma el tipo de **Purchase**.

Tipos de unión

```
type RGB = 'red' | 'green' | 'blue';

let color: RGB = 'red';
color = 'yellow'; // error
color = null;      // error

// Pueden mezclar tipos distintos
type RGBoNulo = 'red' | 'green' | 'blue' | null;
```

Valores que acepta **RGB**:

✓ 'red'

✓ 'green'

✓ 'blue'

✗ 'yellow'

✗ null

Las uniones de literales son perfectas para props con un conjunto cerrado de valores, como el **type** de la alerta.

interface frente a type

Característica	interface	type
Describir la forma de un objeto	Sí	Sí
Ampliar otro tipo	extends	& (intersección)
Tipos de unión ('a' 'b')	No	Sí
Nombrar primitivos, tuplas o funciones	No	Sí
Declaraciones repetidas que se fusionan	Sí	No (error)

```
interface Alumno {  
  nombre: string;  
}  
interface AlumnoErasmus  
  extends Alumno {  
    paisOrigen: string;  
  }  
}
```

En React es habitual usar **type** para las props. Lo importante: ser coherente en el proyecto.

as const y satisfies

as const

```
const configuracion =  
  { tema: 'oscuro' } as const;  
// { readonly tema: 'oscuro' }
```

Tipos literales y propiedades de solo lectura.

satisfies

```
type Tema = 'claro' | 'oscuro';  
const ajustes = {  
  tema: 'oscuro',  
} satisfies { tema: Tema };  
ajustes.tema; // 'oscuro'
```

Comprueba el tipo sin perder el tipo inferido.

La alternativa moderna a enum

```
const Rol = {  
  Admin: 'ADMIN',  
  SoloLectura: 'SOLO_LECTURA',  
  Autor: 'AUTOR',  
} as const;  
  
type Rol = (typeof Rol)[keyof typeof Rol];  
// 'ADMIN' | 'SOLO_LECTURA' | 'AUTOR'  
  
const miRol: Rol = Rol.Admin;
```

¿Por qué evitar enum?

Genera código JavaScript propio: no basta con **borrar** los tipos al compilar.

erasableSyntaxOnly

La plantilla de Vite activa esta opción, que **prohíbe** los enum.

04

El compilador de TypeScript

Cómo comprobar tipos y transpilar con tsc, y cómo configurarlo con tsconfig.json.

Configurar el compilador: tsconfig.json

```
{
  "compilerOptions": {
    "outDir": "dist",
    "target": "esnext",
    "module": "esnext",
    "lib": ["DOM", "esnext"],
    "strict": true,
    "jsx": "react-jsx",
    "moduleResolution": "bundler",
    "noEmitOnError": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

strict

Comprobación más estricta. Por defecto desde TS 6.

jsx: react-jsx

Modo actual: no hace falta importar React.

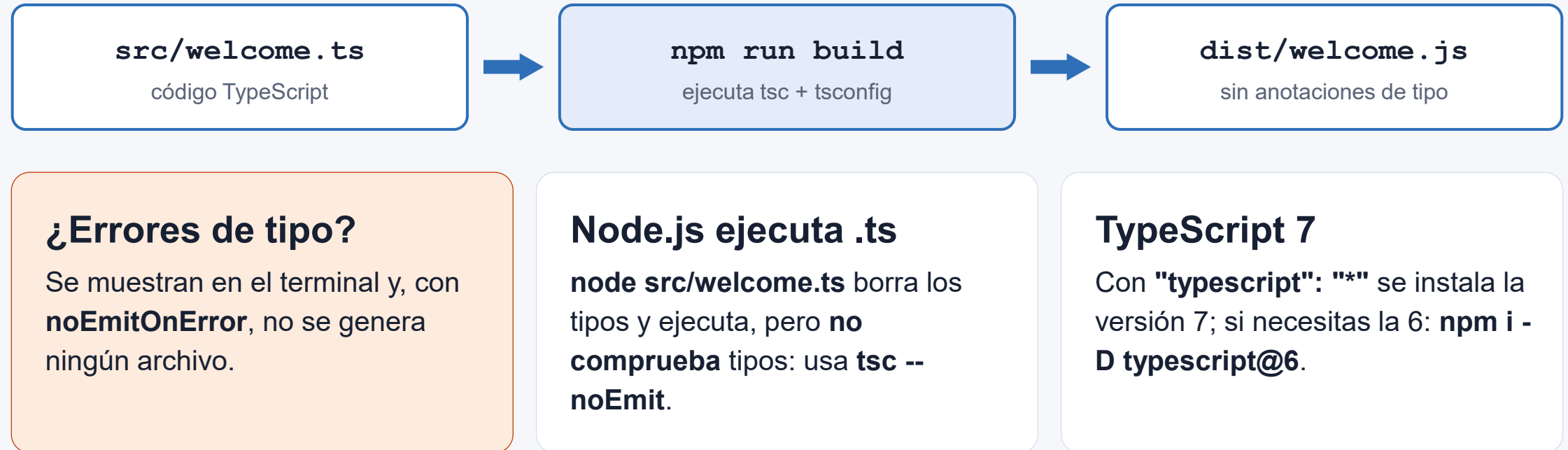
moduleResolution: bundler

Recomendado con Vite. «node» está obsoleto.

noEmitOnError

Si hay errores de tipo, no genera JS.

Del código TypeScript al JavaScript



05

React + TypeScript

Tipamos de forma estricta el componente `Alert`: props, estado y eventos.

Crear el proyecto con Vite

```
npm create vite@latest alert -- --template react-ts  
cd alert && npm i && code . && npm run dev
```

tsconfig.json

solo referencia a los otros dos



tsconfig.app.json

código de src/

tsconfig.node.json

vite.config.ts

verbatimModuleSyntax

Obliga a importar los tipos con **type**.

erasableSyntaxOnly

Prohíbe sintaxis no borrable, como **enum**.

Tipar las props del componente

```
import { useState, type ReactNode } from 'react';

type Props = {
  type?: 'information' | 'warning';
  heading: string;
  children: ReactNode;
  closable?: boolean;
  onClose?: () => void;
};

export function Alert({
  type = 'information', heading,
  children, closable, onClose,
}: Props) { ... }
```

Obligatorias

heading y children.

Unión literal

type solo acepta sus dos valores: detecta erratas y autocompleta.

ReactNode

JSX, cadenas, números...

Importar tipos con la palabra clave type

Incorrecto en la plantilla de Vite:

```
import { useState, ReactNode } from 'react';  
// 'ReactNode' is a type and must be imported using a  
// type-only import when 'verbatimModuleSyntax' is enabled.
```

Correcto:

```
import { useState, type ReactNode } from 'react';
```

Los tipos no existen en ejecución: marcándolos con **type**, basta con borrar esas importaciones al generar JavaScript.

Las props, comprobadas al usar el componente

```
<Alert /> // error: faltan heading y children

<Alert heading="Success">
  ¡Todo va muy bien!
</Alert> // correcto

<Alert type="error" heading="Fallo"> // error: 'error' no es válido
  Algo ha ido mal
</Alert>
```

Corrige el último caso con **type="warning"**: el editor te propone los dos valores válidos.

Tipar el estado con useState

Declaración	Tipo resultante
<code>useState(true)</code>	<code>boolean</code>
<code>useState()</code>	<code>undefined</code>
<code>useState<boolean>()</code>	<code>boolean undefined</code>
<code>useState<boolean>(true)</code>	<code>boolean</code>
<code>useState<Usuario null>(null)</code>	<code>Usuario null</code>

Regla

Comprueba el tipo inferido. Si no es el adecuado, usa el **argumento genérico**.

Caso típico

Estado que empieza vacío y luego recibe datos.

Tipar los eventos

```
function gestionarCambio(
  evento: ChangeEvent<HTMLInputElement>,
) {
  setNombre(evento.target.value);
}

function gestionarEnvio(
  evento: FormEvent<HTMLFormElement>,
) {
  evento.preventDefault();
}

<form onSubmit={gestionarEnvio}>
  <input value={nombre} onChange={gestionarCambio} />
</form>
```

En línea

Si el manejador está en el JSX, el tipo se **infiere**.

Extraído

En una función aparte hay que **indicarlo**:
ChangeEvent, FormEvent, MouseEvent...

Truco

Escribe **onChange={(e) => {}}** y pasa el cursor sobre **e**.

Ampliar las props de un elemento HTML

```
type PropsBoton = ComponentProps<'button'> & {  
  variante?: 'principal' | 'secundario';  
};  
  
export function Boton({ variante = 'principal', ...resto }: PropsBoton) {  
  return <button className={`boton-${variante}`} {...resto} />;  
}
```

```
<Boton variante="secundario" disabled onClick={() => console.log('clic')}>
```

Boton acepta su prop propia y cualquier atributo de <button>, todos con su tipo correcto.

Resumen del tema

Errores pronto

La comprobación estática detecta fallos antes de ejecutar.

unknown > any

unknown obliga a estrechar el tipo antes de usarlo.

Tipos propios

Alias, uniones, intersecciones e interfaces.

as const

Junto a satisfies, alternativa moderna a enum.

tsconfig.json

Configura tsc; TS 7 compila mucho más rápido.

React tipado

Props, estado (useState<T>) y eventos.